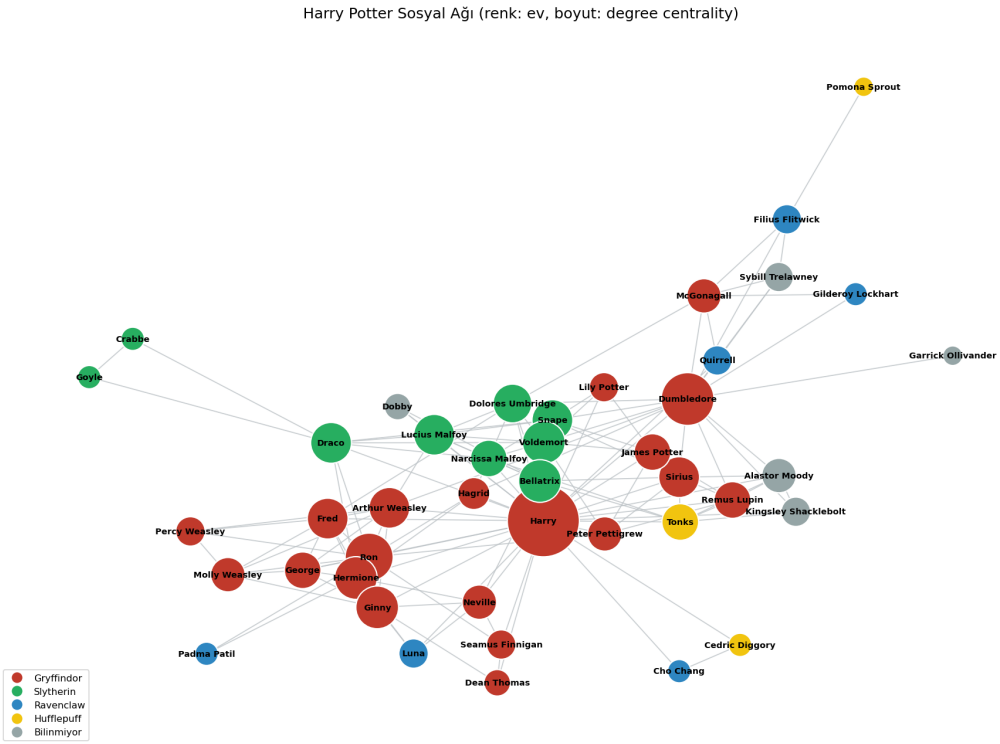
Harry Potter Sosyal Ağ Analizi

Proje ve Kod Rehberi · Mülakat Hazırlık Dokümanı



Graf teorisi · Merkezilik · Topluluk tespiti · Dayanıklılık · Small-world · Yapısal denge

Python · NetworkX · pandas · Matplotlib · pytest

1. Projeyi 30 saniyede anlatmak

Kısa anlatım: "Harry Potter dünyasındaki 43 karakteri düğüm, aralarındaki arkadaşlık, aile, müttefiklik ve düşmanlık ilişkilerini kenar olarak modelleyen bir sosyal ağ analizi yaptım. Ağda kimin merkezi olduğunu, evlerin gerçekten kümelenip kümelenmediğini, ağın kritik karakterler çıkarılınca ne kadar dayandığını ve dostluk/düşmanlık ilişkilerinin yapısal olarak tutarlı olup olmadığını inceledim. Kod, veriyi CSV'den okuyan, doğrulayan ve analizleri test edilmiş bir Python paketi olarak yeniden yapılandırıldı."

Tek bakışta

Konu	Değer
Ağ boyutu	43 düğüm, 142 kenar, 1 bağlı bileşen
İlişki türleri	friend: 40, family: 31, ally: 44, enemy: 27
Yoğunluk / çap	0.157 / 5
Ort. kümelenme	0.632
En merkezi karakter	Harry (tüm merkezilik ölçülerinde), ardından Dumbledore
Kod	src/hpgraph (data, metrics, plots), scripts, tests (23 test), notebook, CI

Ana bulgular

- **Merkezilik:** Harry degree 31, betweenness 0.484. Veri Harry merkezli derlendiği için bu kısmen örneklem yanlılığıdır.
- **Evler:** assortativity r = 0.405 (permütasyon testi p = 0.0005). Evler kümeleniyor ama Louvain toplulukları (modularity 0.490) evlerden (modularity 0.227) daha iyi bir bölümleme veriyor.
- **Small-world:** σ = 3.94. Kümelenme 0.632 (rastgele 0.158), ortalama yol 2.18 (rastgele 2.15).
- **Dayanıklılık:** 5 düğüm silinince en büyük bileşen hedefli saldırıda %81, rastgelede %87. Kesim noktaları: Draco, Dumbledore, Filius Flitwick, Harry.
- **Yapısal denge:** 172 üçgenin 156'i dengeli (%90.7).

2. Problem tanımı ve motivasyon

Sosyal ağ analizi, aktörlerin (kişiler, karakterler) ve aralarındaki ilişkilerin graf olarak modellenip yapısal özelliklerinin incelenmesidir. Bu proje, Ayrık Matematik dersindeki graf teorisi kavramlarını (derece, yol, bağlantılılık, kümelenme) gerçek bir probleme uygular. Harry Potter evreni; tanıdık, yorumlanabilir ve hem dostluk hem düşmanlık içeren zengin bir ilişki yapısı sunduğu için seçilmiştir. Yorumlanabilirlik önemlidir: Snape'in neden köprü olduğunu hikâyeden bilerek sonucu doğrulayabiliriz.

Araştırma soruları

- **S1.** Ağın merkezinde kim var, bilgiyi/ilişkiyi köprüleyen karakterler kimler?
- **S2.** Hogwarts evleri sosyal olarak kümeleniyor mu, yoksa topluluklar evlerden bağımsız mı?
- **S3.** Ağ ne kadar dayanıklı; kritik karakterler çıkarılınca ne olur?
- **S4.** Ağ small-world mü (kısa yollar + yüksek kümelenme)?
- **S5.** Dostluk/düşmanlık ilişkileri yapısal olarak tutarlı mı (Heider dengesi)?

3. Graf teorisi temelleri

Bir graf G = (V, E) düğüm kümesi V ve kenar kümesi E'den oluşur. Burada |V| = n = 43 karakter, |E| = m = 142 ilişkidir. Graf **yönsüz**dür (arkadaşlık karşılıklıdır), kenarlar bir ilişki türü ve 1-3 arası bir ağırlık taşır.

Kavram	Tanım	Bu projedeki karşılığı
Derece (degree)	Bir düğüme bağlı kenar sayısı. Ortalama derece = 2m/n.	Harry 31; ortalama 6.60
Yol / en kısa yol	İki düğümü birleştiren kenar dizisi; en kısa olanın uzunluğu d(u,v).	Ortalama en kısa yol 2.18
Bağlı bileşen	Aralarında yol bulunan düğümlerin maksimal kümesi.	Tek bileşen (43 düğümün hepsi)
Yoğunluk	m / (n(n-1)/2): olası kenarların kaçı var?	0.157
Çap / yarıçap	En büyük / en küçük eksantriklik (bir düğümün en uzak düğüme mesafesi).	5 / 3
Kümelenme katsayısı	Bir düğümün komşularının birbirine bağlı olma oranı: C(v) = 2T(v) / (k(k-1)).	Ortalama 0.632
Geçişlilik	3 × üçgen sayısı / bağlı üçlü sayısı (global kümelenme).	0.386
Kesim noktası	Çıkarılınca bileşen sayısını artıran düğüm.	Draco, Dumbledore, Filius Flitwick, Harry
Köprü	Çıkarılınca bileşen sayısını artıran kenar.	Dumbledore–Garrick Ollivander; Filius Flitwick–Pomona Sprout
k-core	Her düğümün en az k derecesi olduğu maksimal alt graf.	core_number sütunu

4. Veri modeli

Veri iki CSV dosyasındadır. Bu ayrım, veriyi koddan bağımsız hale getirir: yeni karakter eklemek için kod değil CSV değişir.

```
data/characters.csv        name, house
data/relationships.csv     source, target, relation, weight
                           relation in {friend, family, ally, enemy}
                           weight   in {1, 2, 3}   # öznel ilişki yoğunluğu
```

Doğrulama (data.validate) şu hataları yakalar ve hepsini tek mesajda bildirir: yinelenen karakter, geçersiz ev adı, karakter listesinde olmayan düğüm, kendine bağlanan kenar, geçersiz ilişki türü veya ağırlık, yinelenen ilişki çifti (A–B ile B–A aynı kenardır).

Veri güvenilirliği: Veri filmlerden/kitaplardan elle derlenmiş ve öznel bir listedir. Ağırlıklar ölçülmüş değerler (ör. birlikte geçen sahne sayısı) değil, 1-3 arası yorumdur. Mülakatta bunu açıkça söylemek bir zayıflık değil, bilimsel dürüstlüktür.

5. Kod mimarisi

Dosya	Sorumluluk
src/hpgraph/data.py	CSV okuma, doğrulama, build_graph (tüm ilişkiler) ve build_positive_graph (düşmansız)
src/hpgraph/metrics.py	Tüm analizler: özet, merkezilik, topluluk, dayanıklılık, small-world, denge
src/hpgraph/plots.py	Yalnızca çizim; Figure döndürür, hesap yapmaz
scripts/run_analysis.py	Hepsini çalıştırır; docs/images ve results/ üretir
scripts/build_pdf.py	Bu dokümanı results/summary.json'dan üretir
notebooks/HarryPotter.ipynb	Anlatım katmanı; hesapları pakete bırakır
tests/	23 pytest testi: veri doğrulama + metriklerin elle hesaplanmış doğrulukları
.github/workflows/ci.yml	Her push'ta test ve analiz çalıştırır

Tasarım ilkesi: hesap (metrics) ile görselleştirme (plots) ve veri (data) ayrıdır. Böylece her katman bağımsız test edilebilir ve notebook yalnızca bir arayüzdür.

5.1 Graf oluşturma

```
def build_graph(relations=ALL_RELATIONS) -> nx.Graph:
    characters = load_characters()
    relationships = load_relationships()
    validate(characters, relationships)          # hatalı veri burada durur

    G = nx.Graph()
    for row in characters.itertuples(index=False):
        G.add_node(row.name, house=row.house)    # bağlantısız olsa da düğüm eklenir
    for row in relationships.itertuples(index=False):
        if row.relation in relations:            # türe göre filtre (ör. sadece dostça)
            G.add_edge(row.source, row.target,
                       relation=row.relation, weight=int(row.weight))
    return G
```

nx.Graph yönsüz graftır; aynı çifti iki kez eklemek kenarı tekilleştirir. v1'deki Harry–Luna tekrarı sonucu bozmuyordu ama validate artık böyle yinelemeleri hata olarak raporlar.

5.2 Merkezilik tablosu

```
def centrality_table(G):
    table = pd.DataFrame({
        "degree":      dict(G.degree()),
        "strength":    dict(G.degree(weight="weight")),   # ağırlıklı derece
        "closeness":   nx.closeness_centrality(G),
        "betweenness": nx.betweenness_centrality(G),
        "eigenvector": nx.eigenvector_centrality_numpy(G),
        "pagerank":    nx.pagerank(G, weight="weight"),
        "core_number": nx.core_number(G), ...
    })
    return table.sort_values("degree", ascending=False)
```

5.3 Yapısal denge (Heider)

```
sign = {frozenset(e[:2]): (-1 if e[2]["relation"] == "enemy" else 1)
        for e in G_full.edges(data=True)}
for a, b, c in itertools.combinations(G_full.nodes, 3):
    if G.has_edge(a,b) and G.has_edge(b,c) and G.has_edge(a,c):
        prod = sign[{a,b}] * sign[{b,c}] * sign[{a,c}]
        balanced += prod > 0      # çarpım pozitifse üçgen dengelidir
```

5.4 Dayanıklılık simülasyonu

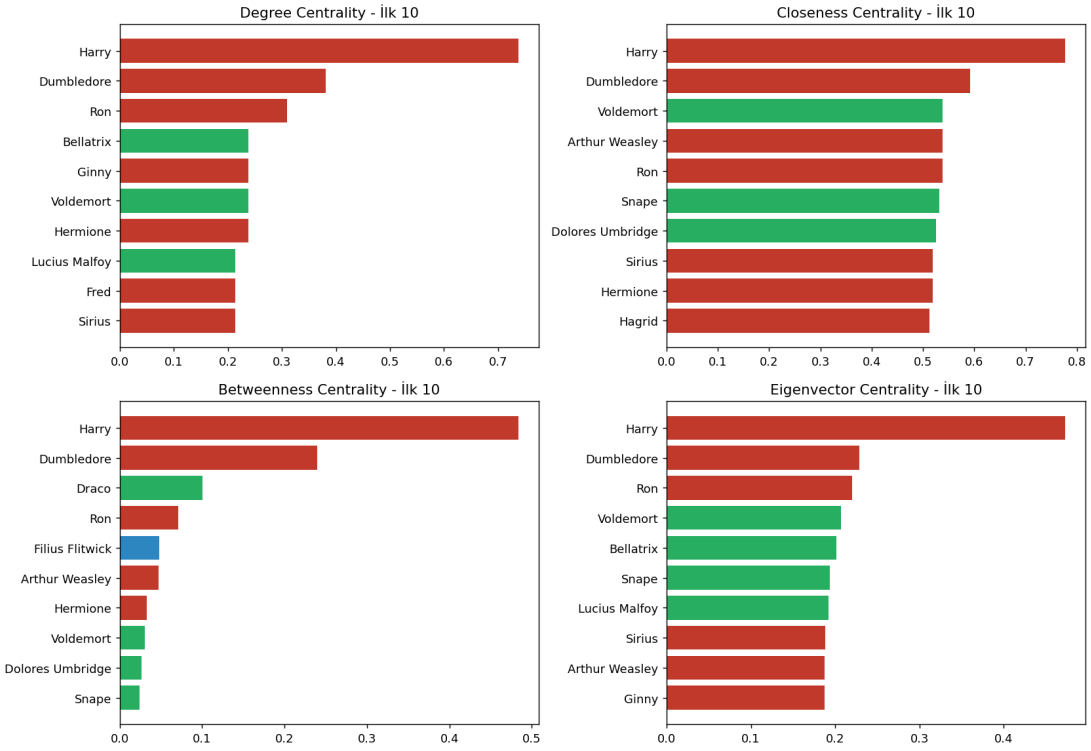
```
order = sorted(G, key=lambda x: (-G.degree(x), x))   # hedefli: yüksek dereceden başla
H = G.copy()
for node in order:
    H.remove_node(node)
    lcc = len(max(nx.connected_components(H), key=len)) / n_total
```

Rastgele senaryoda 200 farklı rastgele sıra denenir ve ortalaması alınır; böylece tek bir şanslı/şanssız sıraya bağlı kalınmaz. Tüm rastgele işlemler seed ile sabitlenmiştir (tekrarlanabilirlik).

6. Merkezilik metrikleri: formül, sezgi, sonuç

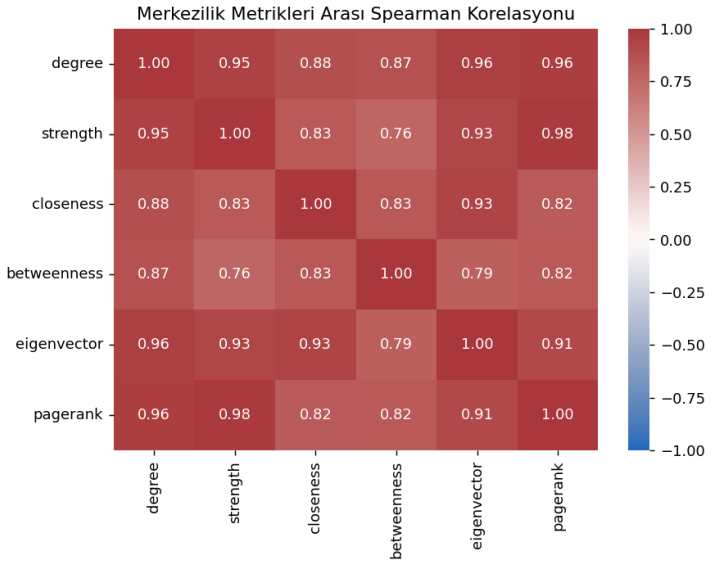
Metrik	Formül	Sorduğu soru	İlk 5
Degree	$C_D(v) = deg(v) / (n-1)$	Kaç doğrudan bağlantısı var?	Harry (31), Dumbledore (16), Ron (13), Hermione (10), Ginny (10)
Closeness	$C_C(v) = (n-1) / \Sigma_u d(v,u)$	Herkese ortalama kaç adımda ulaşır?	Harry (0.778), Dumbledore (0.592), Ron (0.538)
Betweenness	$C_B(v) = \Sigma \sigma_{st}(v) / \sigma_{st}$ (normalize)	En kısa yolların ne kadarı ondan geçer (köprü)?	Harry (0.484), Dumbledore (0.239), Draco (0.100), Ron (0.071), Filius Flitwick (0.048)

Metrik	Formül	Sorduğu soru	İlk 5
Eigenvector	λ x = A x (A: komşuluk matrisi)	Önemli düğümlere mi bağlı?	Harry (0.473), Dumbledore (0.228), Ron (0.220), Voldemort (0.207)
PageRank	PR(v) = (1-d)/n + d Σ PR(u)/deg(u)	Rastgele gezgin nereye uğrar?	Harry (0.102), Dumbledore (0.061), Ron (0.045), Ginny (0.035), Draco (0.035)



Yorum

- **Draco'nun yüksek betweenness'ı:** Crabbe ve Goyle ağa yalnızca Draco üzerinden bağlı; bu yüzden Draco, kendi kümesini ağın geri kalanına bağlayan köprüdür (kesim noktası).
- **Voldemort closeness ve eigenvector'da öne çıkar** çünkü düşmanlık kenarları da ağın parçasıdır; ağ "sosyal etkileşim" ağıdır, yalnızca dostluk ağı değildir.
- **Metrikler yüksek korelasyonlu** (degree ile eigenvector ρ = 0.96, PageRank ρ = 0.96): küçük ve Harry merkezli bir ağda sıralamalar birbirine benzer.



7. Gelişmiş analizler

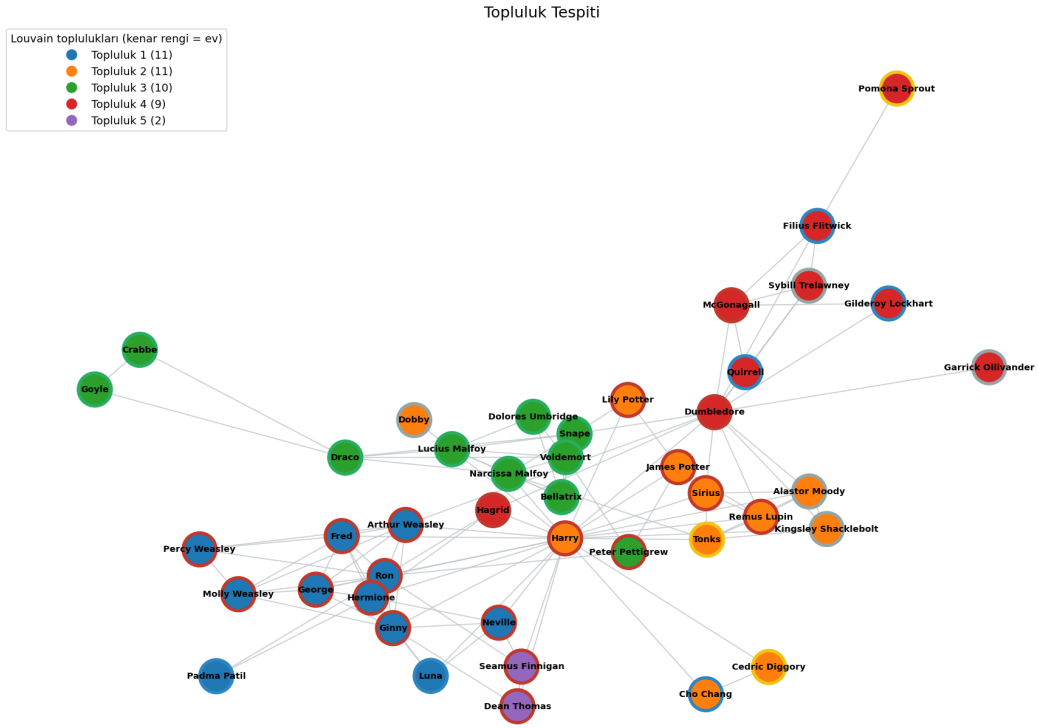
7.1 Topluluk tespiti (Louvain)

Modularity Q, bir bölümlemenin topluluk içi kenar yoğunluğunun rastgele beklentiden ne kadar fazla olduğunu ölçer: $Q = (1/2m)\ \Sigma_{ij}\ [A_{ij} - k_i k_j/2m]\ \delta(c_i, c_j)$. Louvain algoritması, düğümleri komşu topluluklara taşıyarak Q'yu açgözlü biçimde artırır, sonra toplulukları tek düğüme indirip süreci tekrarlar. Seed sabitlenmiştir.

#	Boyut	Üyeler
1	11	Arthur Weasley, Fred, George, Ginny, Hermione, Luna, Molly Weasley, Neville, Padma Patil, Percy Weasley, Ron
2	11	Alastor Moody, Cedric Diggory, Cho Chang, Dobby, Harry, James Potter, Kingsley Shacklebolt, Lily Potter, Remus Lupin, Sirius, Tonks
3	10	Bellatrix, Crabbe, Dolores Umbridge, Draco, Goyle, Lucius Malfoy, Narcissa Malfoy, Peter Pettigrew, Snape, Voldemort
4	9	Dumbledore, Filius Flitwick, Garrick Ollivander, Gilderoy Lockhart, Hagrid, McGonagall, Pomona Sprout, Quirrell, Sybill Trelawney
5	2	Dean Thomas, Seamus Finnigan

Louvain modularity **0.490**, ev bölümlemesi **0.227**. NMI (topluluk-ev benzerliği) = **0.417**: toplulukların evlerle ilişkisi var ama özdeş değil. Topluluklar evden çok *hikâye rolünü* yansıtır.

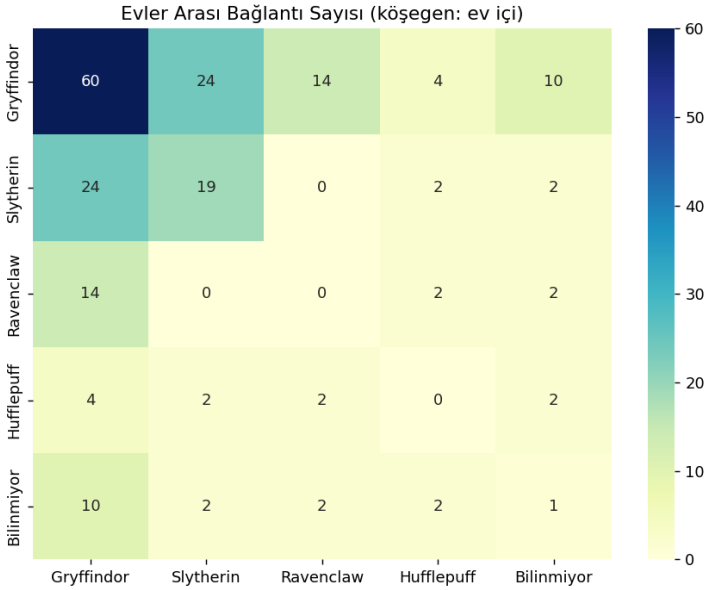
Neden yalnızca dostça ilişkiler? Modularity "kenar = aynı topluluğa aitlik" varsayar. Düşmanlık kenarı (Harry–Voldemort) iki karakteri aynı topluluğa çekmemelidir. Bu yüzden topluluk ve assortativity analizleri düşmansız grafta (build_positive_graph) yapılır.



7.2 Ev assortativity ve permütasyon testi

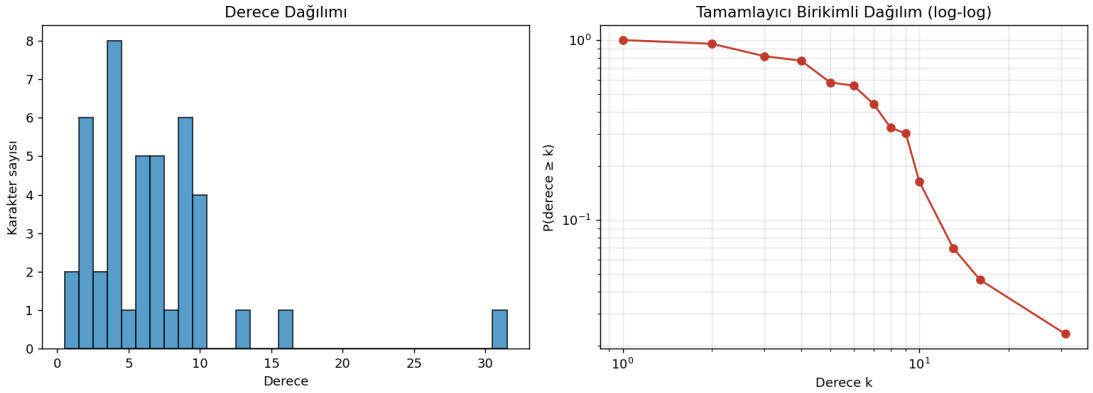
Assortativity katsayısı r, kenarların aynı etiketli düğümler arasında beklenenden ne kadar fazla olduğunu ölçer (+1: tam homofili, 0: rastgele, negatif: zıt eşleşme). Bulunan r = 0.405 anlamlı mı? **Permütasyon**

testi: ev etiketleri düğümler arasında 2000 kez rastgele karıştırılır, her seferinde r yeniden hesaplanır. Gözlenen r'ye eşit veya büyük değerin görülme oranı p-değeridir: **p = 0.0005** (2000 permütasyonda hiçbiri aşmadı; 1/2001 ölçülebilir en küçük değerdir). Yani bu kadar yüksek ev homofilisi tesadüfle açıklanamaz. Parametrik varsayım gerektirmediği için küçük ve bağımlı (kenarlar bağımsız değil) bir ağda uygun bir testtir.



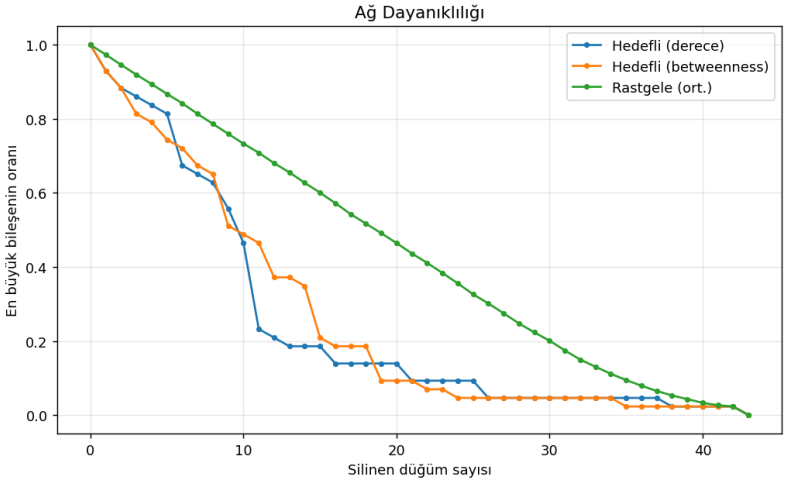
7.3 Small-world analizi

Watts–Strogatz small-world ağlar, rastgele ağlar kadar kısa ortalama yola ve çok daha yüksek kümelenmeye sahiptir. Aynı n ve m ile 100 Erdős–Rényi grafı üretilip ortalama alınır: $\sigma = (C / C_{rand}) / (L / L_{rand})$. Burada C = 0.632, C_{rand} = 0.158, L = 2.185, L_{rand} = 2.155, **σ = 3.94** ($\sigma > 1$: small-world). Not: 43 düğümlü bir ağda L'nin rastgele ağa yakın çıkması zaten beklenen bir durumdur; asıl fark kümelenmede.



7.4 Dayanıklılık

Düğümler tek tek silinirken en büyük bağlı bileşenin oranı izlenir. Gerçek dünya ağlarında tipik olarak hedefli saldırı rastgele hataya göre çok daha yıkıcıdır; çünkü az sayıda hub çok sayıda yolu taşır. Burada 5 düğüm silinince hedefli (derece) saldırıda %81, betweenness'a göre %74, rastgelede %87 kalır.



7.5 İşaretli ağ ve Heider yapısal dengesi

Heider'e göre bir üçgen, kenar işaretlerinin çarpımı pozitifse dengelidir: (+,+,+) "dostumun dostu dostumdur", (+,-,-) "düşmanımın düşmanı dostumdur". Dostça ilişkiler +1, düşmanlık -1 alınmıştır. Sonuç: 172 üçgenin **156**'i dengeli, **16**'i dengesiz (%90.7 denge). Dengesiz üçgenler rastgele değil, hikâyedeki çelişkili karakterlerdedir:

- **Snape**: Dumbledore'un müttefiki ama Harry'nin düşmanı (Harry–Dumbledore–Snape).
- **Peter Pettigrew**: James Potter'ın dostu ama Voldemort'un müttefiki ve Sirius/Remus'un düşmanı.
- **Narcissa Malfoy**: Ölüm Yiyen ailesinde ama Harry'ye yardım eden (yalan söyleyerek).

8. v1'den v2'ye: bulunan hatalar ve düzeltmeler

Proje ilk sürümde çalışıyordu ama veri ve kod kalitesinde sorunlar vardı. Bunları bulup düzeltmek ve *neden* önemli olduklarını söyleyebilmek mülakatta güçlü bir noktadır.

Sorun (v1)	Etkisi	Çözüm (v2)
"Lupin" ve "Remus Lupin" iki ayrı düğümdü (aynı karakter)	Merkezilik değerleri ikiye bölünüyor, düğüm/kenar sayısı yanlıştı	Tek düğüme birleştirildi, test eklendi
Gilderoy Lockhart Gryffindor görünüyordu	Ev matrisi ve renkler yanlış	Ravenclaw olarak düzeltildi
Evi belirsiz karakterlere ev atanmıştı	Uydurma veri	"Bilinmiyor" kategorisi
Harry–Luna kenarı iki kez	NetworkX tekilleştirir, ama veri kirli	Temizlendi; validate yinelemeyi yakalar
Bariz ilişkiler eksikti (Ron–Ginny, Ron–Hagrid, Harry–Neville…)	Cho–Cedric ana ağdan kopuk göründü (yapay 2. bileşen)	Eklendi; ağ tek bileşen
Yalnızca "arkadaşlık" vardı	Düşmanlık yok; sosyal ağ eksik	friend/family/ally/enemy + ağırlık
Aynı veri ve grafik notebook'ta iki kez tanımlıydı, son hücre boştu	Bakım zorluğu, tutarsızlık riski	Veri CSV'ye, kod pakete taşındı
Test, doğrulama, bağımlılık dosyası yoktu	Tekrarlanabilirlik yok	pytest, requirements, pyproject, CI
README sayıları elle yazılmıştı	Veri değişince eskir	Sayılar results/summary.json'dan

9. Sınırlılıklar ve gelecek çalışma

- **Öznel ve küçük veri:** 43 düğüm elle derlendi; istatistikler yol göstericidir, kesin kanıt değildir.

- **Harry merkezli örneklem:** Karakterler Harry'nin etrafından seçildiği için hub etkisi şişirilmiştir (ego-network yanlılığı).
- **Zamansız ağ:** İlişkiler 7 kitap boyunca değişir (Snape, Draco). Zamana bağlı (temporal) ağ daha doğru olur.
- **Öznel ağırlıklar:** Kitaplardan ölçülen birlikte geçme sayısı ile değiştirilebilir.
- **Yönsüz model:** Bazı ilişkiler asimetriktir (Snape→Lily). Yönlü graf düşünülebilir.

Geliştirme fikirleri

- Kitap metinlerinden birlikte geçme (co-occurrence) ağı çıkarıp 100+ karakterlik ağ kurmak.
- Zamana göre (kitap kitap) ağın evrimi; Barabási–Albert ile büyüme modeli karşılaştırması.
- Link prediction (Adamic–Adar, Jaccard) ile eksik ilişkileri tahmin etmek.
- Etkileşimli görselleştirme (PyVis / Plotly) ve Streamlit demosu.

10. Algoritma karmaşıklığı

İşlem	Algoritma	Karmaşıklık
Derece	Komşu sayma	O(n + m)
Closeness	Her düğümden BFS	O(n(n + m))
Betweenness	Brandes algoritması	O(nm) (ağırlıksız)
Eigenvector	Güç iterasyonu / sayısal özdeğer	O(k·m) iterasyon başına O(m)
PageRank	Yinelemeli güç yöntemi	O(k·m)
Kesim noktası / köprü	Tarjan (DFS, low-link)	O(n + m)
Bağlı bileşenler	BFS/DFS	O(n + m)
k-core	Düğüm soyma	O(m)
Louvain	Açgözlü modularity artırma	Pratikte yaklaşık O(n log n)
Yapısal denge (bu kodda)	Tüm üçlülerin taranması	$O(n^3)$: n = 43 için 12 341 üçlü
Dayanıklılık	Her silmede bileşen hesabı	O(n(n + m)) / deneme

Yapısal denge kodu basitlik için tüm üçlüleri tarar ($O(n^3)$). Büyük ağlarda her kenar için komşu kesişimine bakan $O(m \cdot \Delta)$ yöntem kullanılmalıdır; 43 düğümde fark önemsizdir.

11. Mülakat soruları ve örnek cevaplar

Proje ve motivasyon

1. Projeyi kısaca anlatır mısın?

Harry Potter karakterlerini düğüm, ilişkilerini (arkadaşlık, aile, müttefiklik, düşmanlık) kenar olarak modelleyip merkezilik, topluluk tespiti, dayanıklılık, small-world ve yapısal denge analizleri yaptım. Kod, CSV'den veri okuyan, doğrulayan ve 23 testle korunan bir Python paketi.

2. Neden bu konuyu seçtiniz?

Ayrık Matematik dersindeki graf kavramlarını somut bir veriyle uygulamak istedik. Harry Potter ağı yorumlanabilir: sonuçları hikâyeden doğrulayabiliyoruz (Snape köprü, Slytherin ayrı küme). Bu, metriklerin anlamını sınamak için iyi bir laboratuvar.

3. Projede senin katkın neydi?

(Bu soruyu kendi sözlerinle cevapla: ekip üç kişiydi. Örneğin veri modeli, analiz kodu, test, raporlama gibi hangi kısımları yaptığını önceden belirle ve dürüst ol.)

4. v1'e göre ne değiştirdin?

Veri hatalarını düzelttim (Lupin'in çift düğümü, Lockhart'ın evi, eksik ilişkiler), düşmanlık ilişkilerini ekledim, veriyi koddan ayırdım, doğrulama ve test yazdım, analiz derinliğini artırdım (topluluk, dayanıklılık, small-world, denge). Bölüm 8'deki tablo bunu özetler.

Graf teorisi

5. Degree, closeness ve betweenness farkı nedir?

Degree yerel popülerliktir (kaç bağlantı). Closeness küresel yakınlıktır (herkese ortalama kaç adım). Betweenness köprü rolüdür (en kısa yolların kaçı ondan geçer). Çok bağlantılı biri köprü olmayabilir; az bağlantılı biri (Draco) iki kümeyi birbirine bağlayabilir.

6. Eigenvector centrality ile PageRank farkı?

İkisi de "önemli düğümlere bağlı olmak" fikrine dayanır. PageRank rastgele gezginin uğrama olasılığıdır ve sönümleme (d = 0.85) ile ışınlanma içerir; yönlü graflarda ve bağlantısız yapılarda daha kararlıdır. Eigenvector komşuluk matrisinin baskın özvektörüdür ve yönsüz bağlı graflarda iyi çalışır.

7. Closeness bağlantısız grafikte nasıl hesaplanır?

Ulaşılamayan düğümler sonsuz mesafe verir. NetworkX, Wasserman–Faust düzeltmesiyle yalnızca ulaşılabilir düğümleri sayıp bileşen boyutuyla ölçekler. v1'de ağ 2 bileşenliydi ve bu değerleri etkiliyordu; v2'de ağ tek bileşen olduğu için sorun yok.

8. Kesim noktası ve köprü nedir, neden önemli?

Çıkarılınca ağı parçalayan düğüm/kenardır. Tek hata noktasını (single point of failure) gösterir. Burada Harry, Dumbledore, Draco ve Flitwick kesim noktasıdır; Flitwick–Sprout ve Dumbledore–Ollivander köprülerdir.

9. Kümelenme katsayısı ne ifade eder, neden yüksek?

Bir kişinin arkadaşlarının birbirini de tanıma olasılığıdır. Ortalama 0.63, aynı yoğunluktaki rastgele ağdan (≈0.16) çok yüksek; çünkü aileler ve gruplar (Weasleyler, Malfoylar, Hogwarts kadrosu) kapalı üçgenler oluşturur.

10. Small-world nedir, ağınız small-world mü?

Kısa ortalama yol + yüksek kümelenme. $\sigma = 3.94 > 1$ olduğundan evet. Ama 43 düğümde kısa yol zaten kaçınılmaz; ayırt edici olan kümelenmedir. Bunu dürüstçe belirtirim.

Yöntem ve istatistik

11. Modularity nedir? Louvain nasıl çalışır?

Modularity, topluluk içi kenar oranının rastgele (derece korumalı) beklentiden farkıdır; yüksek değer güçlü topluluk yapısı demektir. Louvain, düğümleri komşu topluluğa taşıyarak Q'yu artırır, sonra toplulukları tek düğüme indirip tekrarlar. Sonuç seed'e bağlı olabilir; seed sabitledik. Sınırlılığı: çözünürlük limiti (çok küçük toplulukları görmeyebilir).

12. Neden topluluk tespitini düşmansız grafta yaptınız?

Modularity kenarı "aynı gruba aitlik" kanıtı sayar. Harry–Voldemort düşmanlık kenarı onları aynı topluluğa çekerdi ve anlamsız sonuç verirdi. Tam grafta merkezilik, dayanıklılık ve denge; dostça grafta topluluk ve assortativity analiz edildi.

13. NMI nedir, neden kullandın?

Normalized Mutual Information, iki bölümlemenin birbirinden ne kadar bilgi taşıdığını ölçer (0: ilişkisiz, 1: aynı). Louvain toplulukları ile gerçek evleri karşılaştırmak için kullandım (0.42: kısmi örtüşme). Kendi uygulamamı yazdım ve iki uç durumda test ettim (özdeş = 1, bağımsız = 0).

14. Permütasyon testi nedir, neden t-testi değil?

Etiketleri rastgele karıştırarak "hiçbir ev etkisi yok" null dağılımını üretir ve gözlenen değerin ne kadar uç olduğuna bakar. Ağdaki kenarlar bağımsız olmadığı için klasik testlerin varsayımları sağlanmaz; permütasyon testi dağılım varsayımı gerektirmez.

15. Heider yapısal dengesi nedir?

İşaretli ağlarda üçgenin işaret çarpımı pozitifse dengelidir. Ağımızda %90.7 denge var; dengesizler Snape, Pettigrew ve Narcissa gibi çelişkili karakterlerde toplanıyor, yani metrik hikâyeyle tutarlı.

16. Spearman korelasyonunu neden seçtin?

Merkezilik değerleri farklı ölçeklerde ve doğrusal olmayan dağılımlıdır. Spearman sıralama korelasyonudur; "iki metrik karakterleri aynı sıraya dizer mi?" sorusuna cevap verir ve aykırı değerlere dayanıklıdır.

17. Betweenness'ı neden ağırlıksız hesapladın?

Ağırlıklar öznel (1-3) puanlardır ve güçlü ilişki = kısa mesafe olarak çevrilmesi (1/ağırlık) keyfi bir varsayım olurdu. Yol metriklerini ağırlıksız tuttum, ağırlığı ayrıca *strength* olarak raporladım.

Veri ve doğruluk

18. Veriye ne kadar güvenebiliriz?

Sınırlı. Elle derlendi, öznel ve Harry merkezli. Sonuçlar "bu veri setinde" geçerlidir; Harry'nin merkezi çıkması kısmen örneklem yanlılığıdır. Bunu README'de ve raporda belirttim. Geliştirme olarak kitap metinlerinden co-occurrence ağı çıkarmayı öneriyorum.

19. Ağırlıkları nereden buldun?

Ölçülmedi, 1-3 arası öznel yoğunluk puanıdır (1: sınırlı etkileşim, 3: hikâyenin merkezi). Bu yüzden sonuçların ağırlığa duyarlılığı düşük tutuldu: yol metrikleri ağırlıksız.

20. Bir bilgiyi nasıl doğruladın?

Üç katman: (1) data.validate veri tutarlılığını denetler, (2) metrikler elle hesaplanabilen küçük graflarda test edilir (yıldız grafında betweenness = 1), (3) sonuçlar hikâyeyle karşılaştırılıp mantık kontrolü yapılır.

21. Veri Harry merkezli ise sonuçlar neden anlamlı?

Anlamlılık soruya bağlı. "Harry'nin ağdaki konumu" hakkında değil, yapısal sorular (topluluk, denge, dayanıklılık, ev homofilisi) hakkında çıkarım yapıyoruz. Harry'nin etkisini göstermek için hedefli saldırı senaryosunda en çok o düğümün etkisini görebiliyoruz. Yine de kesin çıkarımlardan kaçınırım.

Mühendislik

22. Kodu neden paket olarak yapılandırdın?

Hesap, çizim ve veri ayrı olunca test edilebilir ve yeniden kullanılabilir. Notebook yalnızca anlatım katmanı. Tek komutla (run_analysis.py) tüm sonuçlar ve grafikler yeniden üretilir; README'deki sayılar elle değil çıktıdan gelir.

23. Nasıl test ettin?

23 pytest testi: veri doğrulamanın hatalı girdileri reddetmesi, graf özellikleri (tek bileşen, Lupin tek düğüm), metriklerin bilinen graflarda elle hesaplanan değerleri, NMI uç durumları, dayanıklılık monotonluğu ve hedefli saldırının rastgeleden kötü olması gibi özellikler. CI her push'ta çalıştırır.

24. Tekrarlanabilirlik nasıl sağlandı?

Tüm rastgele işlemler (layout, Louvain, permütasyon, rastgele graflar) seed ile sabit; bağımlılıklar requirements.txt ve pyproject.toml'da; CI temiz ortamda çalıştırır.

25. Ölçeklenirse ne olur?

Betweenness $O(nm)$, yapısal denge kodum $O(n^3)$. 10 000 düğümde betweenness için örnekleme (k parametresi), denge için komşu kesişimi gerekir. Louvain büyük ağlarda iyi ölçeklenir. Gerekirse graph-tool veya igraph kullanılabilir.

26. Bir şeyi farklı yapsaydın ne olurdu?

Veriyi kitap metninden otomatik çıkarırdım (co-occurrence), zamana göre ağın evrimini incelerdim ve ağırlıklı analizin duyarlılığını (ağırlıkları rastgele bozarak) test ederdim.

Matematiksel sorular

27. n düğümlü bir grafta en fazla kaç kenar olur?

$n(n-1)/2$. Yoğunluk bunun kaçının var olduğudur: 142 / 903 = 0.157.

28. El sıkışma lemması nedir?

$\Sigma \deg(v) = 2m$. Derecelerin toplamı çifttir. Burada ortalama derece $2 \cdot 142/43 = 6.6$.

29. Bir graf ne zaman ağaçtır?

Bağlı ve döngüsüz ise; eşdeğer olarak bağlı ve $m = n-1$. Bizim ağ $m = 142 > 42$ olduğu için çok sayıda döngü içerir (üçgenler kümelenmeyi oluşturur).

30. Yıldız grafında (1 merkez, 4 yaprak) betweenness kaçtır?

Merkez için normalize edilmiş değer 1, yapraklar için 0: tüm yaprak çiftlerinin tek en kısa yolu merkezden geçer. Bu, testimizde kullandığımız elle doğrulama.

12. Projeyi çalıştırma

```
git clone https://github.com/MelihAliCagman/HarryPotterGraph.git
cd HarryPotterGraph
python -m venv .venv && .venv\Scripts\activate     # Linux/macOS: source .venv/bin/activate
pip install -e ".[dev]"

python scripts/run_analysis.py      # grafikler (docs/images) + sonuçlar (results/)
pytest                              # 23 test
python scripts/build_pdf.py         # bu rehberi yeniden üretir
jupyter notebook notebooks/HarryPotter.ipynb
```

Mülakat öncesi kontrol listesi: (1) 30 saniyelik özeti ezberle. (2) Bölüm 8'deki hata/düzeltme tablosunu anlat. (3) Metriklerin formüllerini ve farkını bil. (4) Sınırlılıkları kendin söyle. (5) Kendi katkını net tanımla. (6) Notebook'u çalıştırıp bir kez canlı gez.